

# UniStudio — Interview Preparation

A guide to presenting your project for the **MACH8 / LiveWall – AI Developer Internship** (Stage, Tilburg).

Part 1: how to talk about your project. Part 2: the technical foundations you should understand.

Updated 9 June 2026 with the real vacancy text. Key correction: the employer is **MACH8**, LiveWall's dedicated AI hub — not the general creative agency. This makes your project an even stronger match.

## PART 1 — Your Project, Tailored to the MACH8 AI Developer Internship

### 1.1 The most important thing to understand: MACH8 vs LiveWall

**This role is not really “LiveWall the creative agency.” It sits inside MACH8, LiveWall's AI hub.** The vacancy says: *“MACH8, onze AI hub, helpt organisaties om AI niet alleen te begrijpen, maar vooral toe te passen. We ontwikkelen AI-oplossingen die impact maken — van tools en automatiseringen tot innovatieve producten en experiences.”* (MACH8, our AI hub, helps organisations not just to *understand* AI but above all to **apply** it. We build AI solutions that make impact — from tools and automations to innovative products and experiences.)

Why this matters for you:

- **LiveWall** = a creative digital agency (15 years, ~80 people, offices incl. Tilburg/Amsterdam/Antwerp). Clients: McDonald's, Rituals, KLM, HEMA, Heineken. Their core business is campaigns, activations, games, apps — *not* AI research.
- **MACH8** = their AI hub / division — the “branch” that actually does AI. It builds **applied / generative AI**: tools, automations, agents, proof-of-concepts and AI products for clients.
- Proof they really ship AI: LiveWall/MACH8 built **Airmail** for KLM — an AI-powered newsletter tool used across ~65 markets that cut creation time by ~90%.
- So in the interview, talk **applied AI** (integrating LLMs and models into real products), **not** training neural networks from scratch. That is exactly what your project is.

**Say this and you instantly sound informed:** “I understand this role is with MACH8, your AI hub focused on *applying* AI — building tools, agents and products on top of LLMs and AI APIs. That's exactly what I've been doing with my own project.”

### 1.2 The vacancy in their own words (so you recognise the phrasing)

**The pitch they open with:** *“Wil jij werken met de nieuwste AI-technologie en leren hoe je echte AI-oplossingen bouwt? Dan zoeken wij jou.”* (Do you want to work with the newest AI technology and learn how to build real AI solutions? Then we're looking for you.)

**What you'll do (“Wat je gaat doen”)** — “Als AI Developer stagiair werk je mee aan het bouwen van AI-oplossingen en prototypes...”:

| They wrote (Dutch)                           | Meaning (English)                          |
|----------------------------------------------|--------------------------------------------|
| Meebouwen aan AI-tools, agents en prototypes | Help build AI tools, agents and prototypes |
| Werken met LLM's en AI API's                 | Work with LLMs and AI APIs                 |
| Experimenteren met nieuwe AI-technologie     | Experiment with new AI technology          |

| They wrote (Dutch)                                     | Meaning (English)                         |
|--------------------------------------------------------|-------------------------------------------|
| Ondersteunen bij het ontwikkelen van proof-of-concepts | Support development of proof-of-concepts  |
| Meedenken over technische oplossingen                  | Think along on technical solutions        |
| Samenwerken met engineers en innovators                | Collaborate with engineers and innovators |

**Who they want (“Over jou”):** a curious maker who likes to try things and wants to grow toward an **AI Engineer** role. Student (HBO/WO), preferably technical; curious about AI; learns by doing; analytical and solution-oriented; not afraid to make mistakes and experiment; and — crucially — “*basiskennis van programmeren, interesse in AI en ervaring met side-projects of eigen builds*” (basic programming knowledge, interest in AI, and experience with side-projects or **your own builds**).

**What they offer:** real AI projects, lots of room to learn and experiment, mentoring by experienced engineers, a path to grow into an **AI Engineer** role, €500/month, free lunch, 40 hours/week, Tilburg. Apply via “Solliciteer nu” (or hr@livewallgroup.com).

### 1.3 Why UniStudio is an almost word-for-word match

| What the MACH8 vacancy asks for                    | What UniStudio proves about you                                                                                                             |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| “AI-tools, agents en prototypes”                   | You built an <b>AI agent</b> plus ~18 AI tools/modules — literally tools, an agent, and a working prototype.                                |
| “Werken met LLM's en AI API's”                     | You integrate an <b>LLM (Claude)</b> for vision & planning, and call <b>5 AI APIs</b> (Replicate, fal.ai, FASHN, a local model, Anthropic). |
| “Experimenteren met nieuwe AI-technologie”         | You wired up cutting-edge image/video models (Flux, Kolors, Kling, Wan) into one product.                                                   |
| “Ondersteunen bij proof-of-concepts”               | UniStudio <i>is</i> a working proof-of-concept that replaces several paid SaaS tools.                                                       |
| “ervaring met side-projects of eigen builds”       | UniStudio <b>is</b> your own build — designed, built and deployed by you. This bullet is describing you.                                    |
| “groeien richting AI Engineer”                     | You already integrate AI into real products and reason about architecture — exactly the trajectory they're funding.                         |
| Curious, learns by doing, not afraid to experiment | You taught yourself a full AI stack by building something real, debugging live integration problems.                                        |

**The single best sentence you can say:** “Your vacancy asks for someone who builds AI tools and agents, works with LLMs and AI APIs, and has their own builds. I didn't just learn that — I shipped it. My project is an AI agent that orchestrates LLMs and several generative-AI APIs into one deployed product.”

### 1.4 Your 30-second pitch (rephrased for MACH8)

“I built **UniStudio**, a web app that automates e-commerce product photography with AI. The user uploads a raw product photo and the app can remove the background, generate a scene, put the garment on an AI-generated model, add shadows, upscale, even make a short promo video — all in one place. Technically it's an **AI orchestrator** in Next.js and TypeScript: it doesn't train models, it **applies** them, wrapping an LLM (Claude) and several generative-AI APIs behind one clean, modular system. The centrepiece is an **AI agent** that analyses the image, plans the steps, and runs the independent ones in parallel. It's deployed on Vercel.”

## 1.5 Their application: link a project + 3 bullets on why you fit

LiveWall/MACH8 keep applications light: CV/LinkedIn, a link to a project, and **3 bullets (or a short voice/video)**. Use these — tuned to the vacancy:

- **I have my own AI build.** You ask for “ervaring met eigen builds” — UniStudio is exactly that: an AI agent + ~18 AI tools I designed, built and deployed myself.
- **I work with LLMs and AI APIs daily.** I orchestrate Claude plus five generative-AI APIs into one product — the exact “werken met LLM's en AI API's” in your vacancy.
- **I learn by doing and I'm not afraid to experiment.** I taught myself the whole stack by shipping something real and debugging live AI-integration problems — and I want to grow into an AI Engineer.

## 1.6 How to talk honestly about building with an AI assistant

MACH8 literally builds AI tools and uses AI daily — so using an AI coding assistant is a **plus**, not something to hide. But you must **own and understand** the result.

“I built this working closely with an AI coding assistant — which fits how MACH8 works. My role was the **architect and decision-maker**: I chose the structure, picked the providers, defined the data flow, debugged the integration problems, and reviewed every change. The AI sped up the typing; I own the design and understand how every part works.” Then prove it with Part 2.

## 1.7 Likely questions & strong answers

### “What do you know about us / this role?”

“This is with **MACH8**, LiveWall's AI hub. LiveWall is a creative digital agency; MACH8 is the part that **applies** AI — building tools, agents, automations and AI products for clients, like the Airmail tool for KLM. The internship is about building real AI solutions with LLMs and AI APIs and growing toward an AI Engineer role.” (*This alone puts you ahead of most candidates.*)

### “Walk me through your project's architecture.”

Three layers, same pattern everywhere: **UI** (React panel + a hook calling `fetch`) → **API route** (validates, runs the logic) → **provider client** (talks to the AI service). The result URL returns, optionally saves to the DB, and updates client state.

### “Did you train any models?”

“No — and that's the point of applied AI. I **integrate** models. The intelligence lives in providers (Flux, Kolours, Kling, and Claude for vision and planning). My value is the **orchestration**: validation, choosing the right model per case, chaining steps, handling async jobs, errors and cost. That's how AI actually ships in a product.”

### “Tell me about the AI agent.”

A 5-step engine: upload → analyse the image (Claude vision) → plan the steps (Claude) → user edits the plan → execute. Independent steps run **in parallel** with `Promise.all`, saving 25–35s per photo. It respects a **budget tier** (free / economy / premium) and picks providers accordingly. This maps directly to “AI-tools, agents en prototypes.”

### “Hardest problem you solved?”

Two: (1) providers behave differently — some synchronous, some an async queue you **submit then poll**; I handled timeouts (Vercel kills a request at 60s, so slow video routes get 300s). (2) API keys from the environment had a trailing newline causing **401 errors** — fix was `.trim()` in every client.

### “What would you build here / where do you want to grow?”

“I want to grow into an AI Engineer. I'd love to work on AI agents and proof-of-concepts for clients — the same kind of orchestration I built, but for real briefs. I'm comfortable going from idea to a deployed prototype fast.”

### 1.8 Smart questions to ask MACH8

- “What kind of AI projects is MACH8 working on right now — more internal tools, or client-facing AI products?”
- “Which models and AI APIs does the team use day to day?”
- “What does the growth path from intern to AI Engineer look like here?”
- “Was Airmail for KLM built by this team? What did that involve technically?”
- “Will I work mostly on agents/LLM features, or across the whole stack?”

### 1.9 Quick fact sheet

| Thing        | Fact                                                                                     |
|--------------|------------------------------------------------------------------------------------------|
| Employer     | MACH8 — LiveWall's AI hub (applied / generative AI)                                      |
| Role         | AI Developer intern, growing toward AI Engineer                                          |
| They want    | AI tools/agents, LLMs & AI APIs, proof-of-concepts, own builds, learns by doing          |
| Terms        | €500/month, 40h/week, Tilburg, free lunch, engineer mentoring                            |
| Your project | UniStudio: AI agent + ~18 AI tools, orchestrating an LLM + 5 AI APIs, deployed on Vercel |
| Stack        | Next.js 16, React 19, TypeScript, Zustand, Prisma+PostgreSQL, Tailwind, Sharp, Fabric.js |
| Core idea    | An <i>orchestrator</i> that <b>applies</b> AI (doesn't train it) — matches MACH8 exactly |

## PART 2 — Technical Foundations (the *why* behind every choice)

For each technology: **what it is**, a little **syntax**, and **why you chose it**. Understand these and you can answer almost any “how does it work” question. Don't memorise — understand.

### 2.1 The mental model of the whole app

| Browser (UI)                | Server (Next.js API)                             | Outside world              |
|-----------------------------|--------------------------------------------------|----------------------------|
| -----                       | -----                                            | -----                      |
| React page / panel          | route.ts handler                                 | External AI provider       |
| -> hook (manages state)     | -> validate input                                | (Replicate / fal / Claude) |
| -> fetch('/api/bg-remove')  | -> processing logic                              |                            |
|                             | -> call provider client --> AI runs, returns URL |                            |
| update Zustand store <----- | send JSON response <-----                        | result                     |

Every feature is this same flow. Saying that shows you see the *pattern*, not just the code.

### 2.2 Next.js + the App Router

**What:** Next.js is a framework built on React. The App Router means your folder structure *is* your routing: a folder with a page .tsx becomes a web page; a folder with a route .ts becomes an API endpoint.

```
src/app/editor/page.tsx      -> the web page at /editor
src/app/api/bg-remove/route.ts -> the API endpoint at /api/bg-remove
```

#### Syntax of an API route:

```
export async function POST(request: Request) {
  const body = await request.json();          // read the incoming data
  // ...do the work...
  return Response.json({ url: resultUrl });    // send JSON back
}
```

**Server vs Client components:** by default components render on the server (fast, no secrets leak). A file that needs interactivity starts with the directive "use client" at the top.

**Why you chose it:** one codebase for front *and* back end, great Vercel deployment, and file-based routing keeps a big project organised.

### 2.3 React + components

**What:** React builds UI out of **components** — reusable functions that return markup (JSX). The UI is a function of **state**: when state changes, React re-renders.

```
function UploadButton({ label }) {           // props = inputs to a component
  const [busy, setBusy] = useState(false);    // state = data that can change
  return <button onClick={() => setBusy(true)}>{label}</button>;
}
```

**JSX** is HTML-like syntax inside JavaScript. **Props** are inputs from a parent. **Hooks** are functions starting with use: useState (changing data), useEffect (side-effects/cleanup), and custom hooks (yours: useImageProcessing, useAgentPipeline) that bundle reusable logic.

**Why:** the component model lets you reuse UI pieces (your ~18 panels share buttons, dropzones, sliders) and keeps the interface predictable.

### 2.4 TypeScript

**What:** JavaScript with **types**. You declare the shape of your data and the compiler catches mistakes *before* running.

```
type TryOnRequest = {
  garmentUrl: string;
  modelUrl: string;
  provider: "kolors" | "fashn" | "idm-vton"; // only these 3 allowed
};
```

**Why:** in a 37k-line app with many providers, types are documentation that can't go stale and prevent “undefined is not a function” bugs.

## 2.5 State management with Zustand

**What:** a small library for **global state** — data many components share (gallery history, brand colours, video settings).

```
const useGalleryStore = create((set) => ({
  images: [],
  addImage: (img) => set((s) => ({ images: [...s.images, img] })),
}));
const images = useGalleryStore((s) => s.images); // in a component
```

**Why Zustand over Redux:** far less boilerplate, no providers wrapping the app, easy localStorage persistence. **Smart detail:** the brand store uses `partialize` to keep base64 image data *out* of localStorage (it caps at ~5 MB and big blobs would crash it).

## 2.6 API routes, REST and fetch

**What:** front-end and back-end talk over HTTP. The browser sends a request with `fetch`; the route responds with JSON.

```
const res = await fetch('/api/bg-remove', {
  method: 'POST',
  body: JSON.stringify({ imageUrl }),
});
const data = await res.json(); // { url: "..."} }
```

**Why this split:** secrets (API keys) and heavy work stay on the **server** — never exposed to the browser.

## 2.7 The orchestrator concept + async patterns (your key topic for MACH8)

**What:** the back-end's real job is to call external AI and manage the waiting. Two patterns:

- **Synchronous** (Replicate): call, wait, get the result.
- **Async queue** (fal.ai / FASHN): *submit* a job, get an id, then **poll** until the result is ready.

```
const job = await fal.submit(...); // 1. submit
while (!done) {
  const status = await fal.status(job.id); // 2. poll: 'is it done yet?'
  done = status.completed;
}
```

**Parallel execution** — the trick in your AI agent. Independent steps run at the same time:

```
const [noBg, model] = await Promise.all([
  removeBackground(image),
  createModel(prompt),
]); // saves 25-35 seconds
```

**Why it matters:** this *is* applied-AI engineering — the exact skill MACH8 hires for. It shows you understand promises, not just `await` on one line.

## 2.8 Database: Prisma + PostgreSQL (and why it's optional)

**What:** Prisma is an **ORM** — you describe tables in a schema and it generates type-safe functions, so you write JavaScript instead of SQL.

```
model ProcessingJob { id String @id; operation String; cost Float }
await prisma.processingJob.create({ data: { operation: "bg-remove", cost: 0.004 } });
```

**The clever design choice:** the database is **optional**. Every DB call is null-guarded — if `DATABASE_URL` isn't set, the app skips saving and keeps working. **Why:** store history and per-job cost without forcing every user to set up a database.

## 2.9 Styling: Tailwind CSS

**What:** utility-first CSS — you style by composing small classes in the markup.

```
<button class="bg-blue-600 text-white px-4 py-2 rounded">Save</button>
```

**Why:** fast to build, consistent spacing/colours, no jumping between CSS files.

## 2.10 Free compute: Sharp (server) & WASM (browser)

**What:** not everything needs a paid AI call. **Sharp** runs on your server for colour/brightness/shadows — **free**. Background removal can run **in the browser** via WebAssembly — also free, and the image never leaves the user's machine.

**Why mention it:** it shows **cost-awareness** — you only spend on AI when it's genuinely needed. “Free where possible, paid where it counts.”

## 2.11 The canvas editor: Fabric.js

**What:** Fabric.js sits on top of the HTML `<canvas>` and gives you objects you can move, scale, layer and add text to — a mini Photoshop. This is your “Smart Editor.”

## 2.12 Environment variables & secrets

**What:** API keys live in environment variables (`process.env.REPLICATE_API_TOKEN`), never in code, never sent to the browser.

```
const key = process.env.FAL_KEY?.trim(); // .trim() removes a trailing newline
```

**The war story:** keys pulled from Vercel had a hidden newline causing 401s. The fix was `.trim()` in every client — shows you debug from symptom to root cause.

## 2.13 Deployment, Git & Vercel

**What:** you commit with Git; pushing to `main` triggers Vercel to build and deploy automatically to a public URL. Slow routes (video, avatars) get `maxDuration: 300` in `vercel.json` because Vercel cuts requests at 60s by default.

## 2.14 Mini-glossary (terms they might drop)

| Term                    | Plain meaning                                                                                     |
|-------------------------|---------------------------------------------------------------------------------------------------|
| Applied / Generative AI | Using/integrating ready-made AI models to build products (MACH8's focus) — not training new ones. |
| LLM                     | Large Language Model (e.g. Claude) — you use it for vision analysis and planning.                 |

| Term                    | Plain meaning                                                                                          |
|-------------------------|--------------------------------------------------------------------------------------------------------|
| AI agent                | Software that uses an LLM to decide and run a sequence of steps toward a goal (your flagship feature). |
| AI API                  | An endpoint you send data to and get an AI result back (Replicate, fal.ai, Anthropic).                 |
| Orchestration           | Coordinating multiple AI services/steps into one workflow — your app's core job.                       |
| Proof-of-concept        | A small working build that proves an idea is feasible (what an internship project often is).           |
| Polling                 | Repeatedly asking “is it done yet?” until an async job finishes.                                       |
| Async / await / Promise | Tools to wait for slow work (a network call) without freezing the app.                                 |
| Component / State       | A reusable piece of UI / the data that drives what the UI shows.                                       |
| ORM                     | A tool to query a database with code instead of raw SQL.                                               |
| WASM                    | WebAssembly — near-native code in the browser (your free bg-removal).                                  |
| CI/CD                   | Automatic build & deploy when you push code (your Vercel setup).                                       |

**Final advice:** Lead with the MACH8 framing (applied AI, tools and agents, your own build). Speak in *problems and decisions*, not just tool names — “I chose X *because* Y.” Anchor answers to real moments: the parallel-execution speed-up, the 401 newline bug, the optional database. Those stories make an intern sound like a builder. Veel succes!

— Want this in Dutch or Spanish, or as slides? Just ask.